

Don't Melt Your Cache: Low-Associativity with Heat-Sink

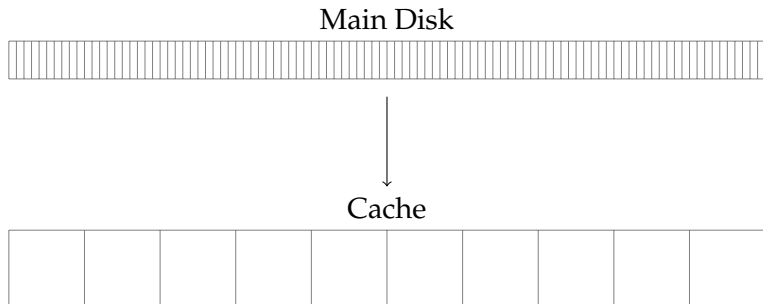
Michael A. Bender, Alex Conway, **Daniel DeLayo**,
Martin Farach-Colton, Jaehyun Han, Linfeng He,
Rob Johnson, Sudarsun Kannan, William Kuszmaul,
Donald Porter, Evan West

Stony Brook University

2025

WHAT A CACHE IS.

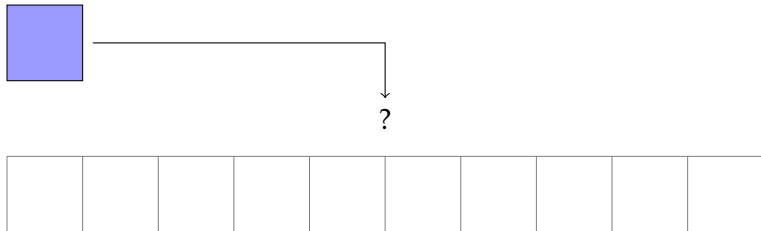
- ▶ Speeds up accesses to something larger and slower.
- ▶ But, only stores a subset of elements.
- ▶ Data must enter the cache to be accessed.



For the purposes of this talk, we'll visualize just the cache.

DATA ENTERS THE CACHE, BUT WHERE?

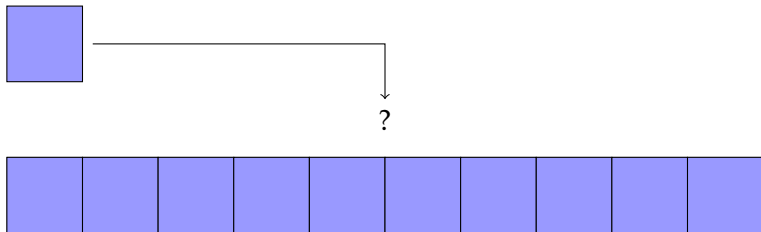
An element is accessed and therefore must enter the cache.
Where do we put it?



We have to consider:

DATA ENTERS THE CACHE, BUT WHERE?

An element is accessed and therefore must enter the cache.
Where do we put it?

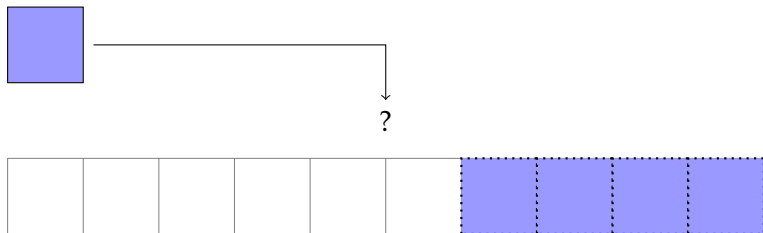


We have to consider:

- What we have to evict, if the cache is full.

DATA ENTERS THE CACHE, BUT WHERE?

An element is accessed and therefore must enter the cache.
Where do we put it?



We have to consider:

- ▶ What we have to evict, if the cache is full.
- ▶ Where the hardware allows each element to go. We'll represent this with a dotted border.

This hardware restriction speeds up lookup and reduces cost.

THREE “TYPES” OF ASSOCIATIVITY BY PICTURE.

Definition.

Associativity: The number of slots an element may hash¹ to.

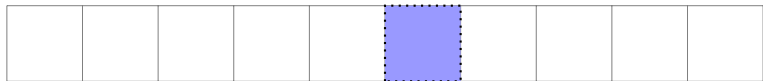
Fully associative– Every element can go anywhere.



d -associative– Each element hashes¹ into a set of d slots.



Direct mapped– Each element hashes¹ into 1 slot.



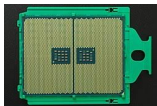
¹ Or otherwise deterministically maps

LOW ASSOCIATIVITY IS EVERYWHERE.

Intel Pentium P5
2-way associative



AMD Milan
8,8,16-way associative



Intel Sky Lake
8,4,16-way associative



Ryzen 5
8,8,16-way associative



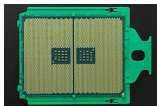
Data from wikichip and wikipedia. Images from wikipedia.

LOW ASSOCIATIVITY IS EVERYWHERE.

Intel Pentium P5
2-way associative



AMD Milan
8,8,16-way associative



Intel Sky Lake
8,4,16-way associative



Ryzen 5
8,8,16-way associative



Observation.

16 is far less than 1 million (64MiB).

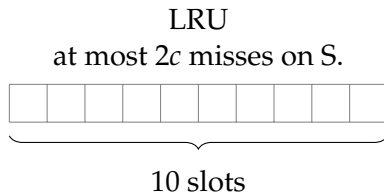
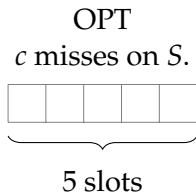
CLASSICAL RESULT: LRU IS $(2, 2)$ -COMPETITIVE.

Definition.

An (α, β) -competitive policy:

- ▶ when given β times more space.
- ▶ incurs at most α times more cache misses.

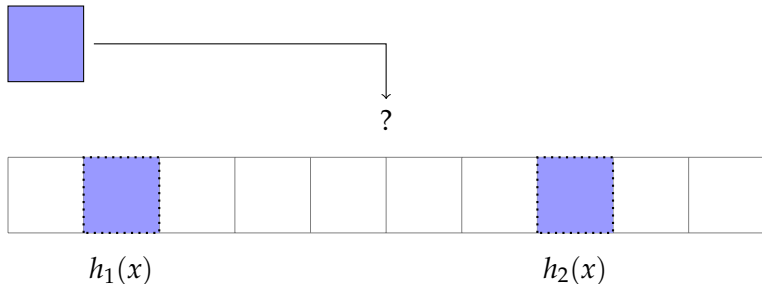
Seminal (fully-associative) result: [Sleator and Tarjan, 1985].
Least Recently Used eviction is $(2, 2)$ -competitive with OPT.



TALK SETUP

Simple Setting For Most of This Talk:

- ▶ Associativity of 2.
- ▶ Each element hashes to 2 slots, $h_1(x)$ and $h_2(x)$.

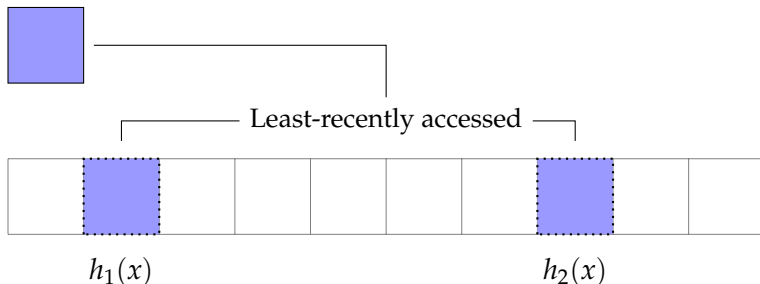


Goal: Design an eviction policy that is $(O(1), O(1))$ -competitive with fully-associative OPT.

THE DOWNFALL OF LRU

The Obvious Algorithm, 2-LRU:

Evict whichever element is least-recently accessed.

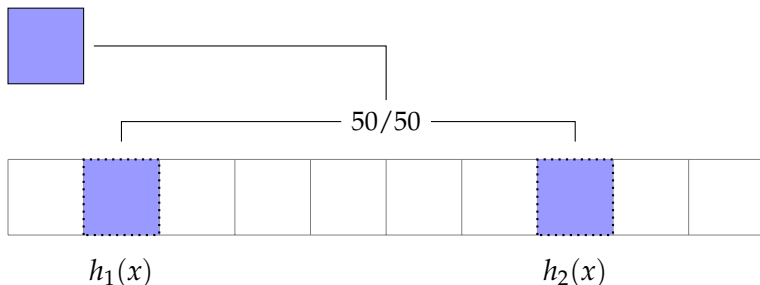


Theorem: 2-LRU is **not** $(O(1), O(1))$ -Competitive.

BUT... 2-RANDOM WORKS

Alternative Strategy, 2-Random:

Evict a *random* element out of the two.

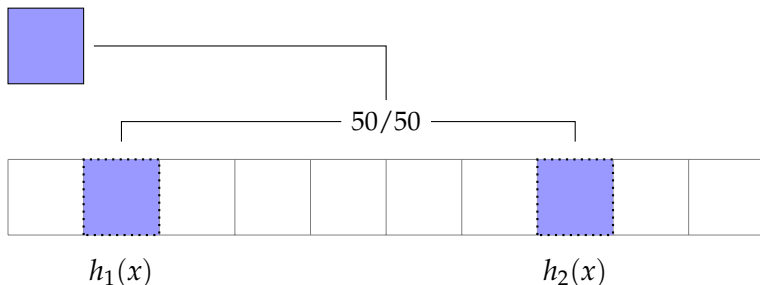


Theorem: 2-Random is $(O(1), O(1))$ -Competitive!

BUT... 2-RANDOM WORKS

Alternative Strategy, 2-Random:

Evict a *random* element out of the two.

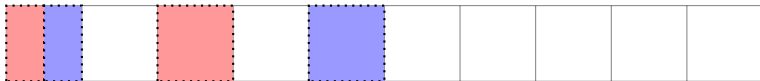


Theorem: 2-Random is $(O(1), O(1))$ -Competitive!

What gives?

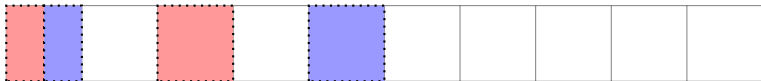
SETTING 2-LRU UP FOR FAILURE.

We have two elements, Red and Blue. They hash to $(0,2)$ and $(0,4)$, respectively.

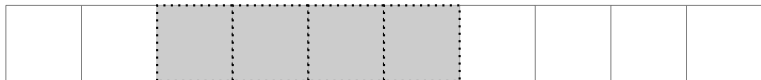


SETTING 2-LRU UP FOR FAILURE.

We have two elements, Red and Blue. They hash to (0,2) and (0,4), respectively.

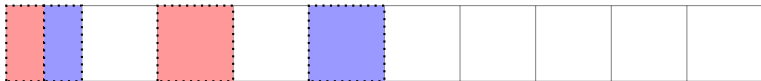


Let's add a set of 2 elements that hash to (2,3) and (4,5).
Call these elements the Stationary elements.

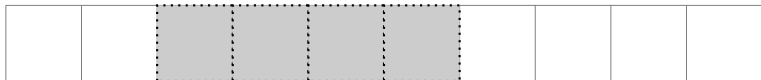


SETTING 2-LRU UP FOR FAILURE.

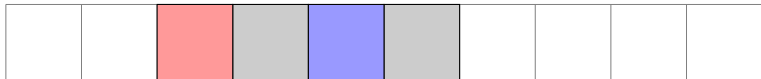
We have two elements, Red and Blue. They hash to (0,2) and (0,4), respectively.



Let's add a set of 2 elements that hash to (2,3) and (4,5).
Call these elements the Stationary elements.



Important: All 4 elements can easily fit in cache together.



2-LRU CAUSES MANY MISSES.

Let's begin by accessing the Stationary elements.



2-LRU CAUSES MANY MISSES.

Let's begin by accessing the Stationary elements.



Request Red and the Stationary elements again.



2-LRU CAUSES MANY MISSES.

Let's begin by accessing the Stationary elements.



Request Red and the Stationary elements again.



Request Blue and the Stationary elements again.



2-LRU CAUSES MANY MISSES.

Let's begin by accessing the Stationary elements.



Request Red and the Stationary elements again.



Request Blue and the Stationary elements again.



Request Red and the Stationary elements again.



WHERE 2-RANDOM GOES RIGHT.

Let's begin by accessing the Stationary elements.



Request Red and the Stationary elements again.



WHERE 2-RANDOM GOES RIGHT.

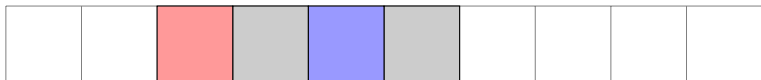
Let's begin by accessing the Stationary elements.



Request Red and the Stationary elements again.



Request Blue and the Stationary elements again.



Observation: The elements randomly orient themselves, driven by contention.

THE SITUATION

Bad News: 2-LRU is not $(O(1), O(1))$ -competitive with fully associative OPT.

Good News: 2-RANDOM is $(O(1), O(1))$ -competitive with fully associative OPT.

BIGGER QUESTION: WHAT IS THE COST OF LOW ASSOCIATIVITY?

Question: Can we design a low-associativity algorithm that is *very nearly* as good as fully associativity LRU?

Answer in Paper: Yes!

We introduce **Heat-Sink LRU**, a d -associative algorithm that is $(1 + o(1), 1 + o(1))$ -competitive with **fully-associative LRU**.

CONCLUSION AND FUTURE WORK.

Conclusions:

- ▶ LRU on limited associativity may not perform well.
- ▶ Small amounts of randomness can obtain asymptotically better performance.

Future Work:

- ▶ Remove mild distributional assumption on the hash functions.
- ▶ How do HEAT-SINK LRU and similar policies perform on real-world workloads or in hardware?